

Integrity Mechanisms for LLM-Written Knowledge Graphs

Starting point: the supernode problem in graph-based agent memory

Agentic Systems Labs

Draft v1, July 29, 2026

1 Vision

Graph databases are becoming the memory substrate for AI agents. Systems like Graphiti (Zep), HippoRAG, and Mem0’s graph variant store what an agent knows as a knowledge graph, built automatically by LLMs reading conversations and documents, and queried through graph algorithms such as Personalized PageRank at every agent step.

This shift breaks a foundational assumption. Classical graph databases were designed for a trusted, careful writer: a human data engineer who deduplicates entities, validates inserts, and maintains a stable schema. Every invariant the engine relies on (one node per entity, edges are true facts, relation types are consistent) was guaranteed outside the database. LLM extraction removed the human but kept the assumptions. The result is a class of structural, temporal, semantic, and concurrency pathologies that the engine faithfully indexes and that multi-hop retrieval then amplifies, because every error lies on exponentially many paths.

The long-term goal of this research program is an “immune system” for machine-written graphs: integrity constraints for probabilistic writers, analogous to `UNIQUE` and `FOREIGN KEY` in relational databases, enforced by mechanisms in the storage and retrieval engine rather than by human curation. Each pathology defines a constraint; each constraint needs a detection metric, a write-time enforcement mechanism, a background repair process, and a benchmark demonstration.

2 The Pathology Catalog (Scope of the Program)

Structural (the graph’s shape is wrong):

- **Supernodes:** LLM extraction systematically creates generic hub nodes (“user”, “meeting”) connected to everything; retrieval mass pools in them and leaks across unrelated facts. *This proposal’s focus.*
- Duplicate entities: one real-world entity fragmented across several nodes; facts scatter, retrieval finds shards.
- Orphan fragments: missed links leave disconnected islands invisible to walk-based retrieval.
- Hallucinated edges: invented relations asserted with the same confidence as true ones.

Temporal (the graph was true once, not now):

- Stale facts: contradictions do not close old facts’ validity windows; state-of-the-art frameworks score below 10% on implicit conflict handling (STALE benchmark).
- Cascade blindness: an invalidated fact’s downstream implications stay live.
- Out-of-order ingestion breaks temporal supersedence logic (documented in Graphiti).

Semantic (meaning drifts):

- Schema drift: `works_at` vs. `employed_by` vs. `job` as three edge types for one meaning.
- Granularity chaos: atomic facts and paragraph summaries coexist as peers.
- Provenance collapse: who said a fact, when, and how trustworthy is flattened away; agents conflate their own inferences with user statements.

Concurrency (many writers):

- No consistency model for multi-agent shared memory exists; concurrent write ordering and visibility are formally undefined in the literature.
- Supernodes double as write hotspots in agent fleets.

The program addresses these through one recurring question: *what invariant did the human curator silently guarantee, and how does the engine guarantee it now?*

3 Work Package 1: The Supernode Problem (This Proposal’s PoC)

3.1 Problem

Retrieval in graph-based agent memory runs Personalized PageRank (PPR) from query-matched seed nodes: probability mass flows along edges, and high-mass nodes are retrieved. Supernodes break this in two ways:

1. **Accumulation:** a hub receives mass from many paths and ranks high for every query, query-independently. Systems retrieve content attached to generic nodes regardless of relevance (an independent Graphiti-vs-Mem0 benchmark observed exactly this: “asking about work returns childhood trauma”).
2. **False bridging:** two unrelated facts that both connect to a generic hub acquire nonzero graph proximity; the walk manufactures multi-hop “evidence” where no semantic relation exists.

LLM extraction makes this worse than in curated graphs: coreference collapses everything onto a “user” node, and type-level entities (“meeting”, “email”) are instantiated once and linked everywhere, producing heavier-tailed degree distributions than human-built KGs.

Existing mitigation is seed-side only: HippoRAG down-weights common phrases as walk entry points (its “node specificity”). Nothing corrects mass pooling or bridging mid-walk. That is the gap.

3.2 Proposed Mechanism (Three Composable Layers)

Layer 1: Transparent hubs (read path). Modify the PPR transition kernel so that mass can pass through a suspicious hub but does not pool in it, and inbound flow to such hubs is damped:

$$P'_{ij} \propto A_{ij} c(j), \quad c(j) = \frac{1}{\log(1 + \deg(j))^\beta}$$

with row renormalization; $\beta = 0$ recovers vanilla PPR. Suspicious hubs are excluded from scoring (corridors, not destinations).

Layer 2: Semantic degree via relation-type entropy (core contribution). Raw degree is the wrong signal: “Acme Corp” with 200 `employs` edges is a legitimate hub; “meeting” with

200 heterogeneous edges is junk. Judge hubs by the entropy of their edge-type distribution:

$$H(v) = - \sum_r p_r(v) \log p_r(v)$$

$$\text{leakiness}(v) = \text{deg}(v) \cdot H(v)$$

High degree with high entropy is punished hard; high degree with low entropy is left mostly alone. Entropy-weighted degree normalization for retrieval over LLM-built graphs does not exist in the literature. It is cheap to compute and incrementally maintainable on inserts.

Layer 3: Write-time facet splitting (engine mechanism). When a node’s leakiness crosses a threshold at ingestion, split it into typed facet nodes (`user` $\rightarrow$ `user#work`, `user#health`, `user#prefs`), clustering its edges by relation type and neighbor embedding, connected by damped spine edges. Queries seed into the relevant facet and stay there; both accumulation and bridging are removed structurally rather than numerically. Facet boundaries additionally align with access-control boundaries (e.g. gating a health facet), connecting this work package to the concurrency and governance packages later in the program.

Layers 1–2 constitute a retrieval-kernel contribution; Layer 3 makes it a graph-engine contribution. In the constraint framing: the enforced invariant is “*no node may accumulate query-independent retrieval mass*”, with leakiness as the detection metric, facet splitting as write-time enforcement, and the damped kernel as read-time repair.

3.3 Evaluation

- **Systems:** swap the kernel into HippoRAG 2 (document QA) and Graphiti (agent memory; its node-distance reranker shares the disease).
- **Benchmarks:** MuSiQue, 2WikiMultiHopQA, HotpotQA (Recall@5, F1/EM) against HippoRAG 2’s published numbers; LongMemEval and LoCoMo through Graphiti.
- **Motivating diagnostics:** degree distributions of LLM-built vs. curated graphs; fraction of PPR mass through top-1% leakiness nodes; hub retrieval frequency under real vs. shuffled queries (query-independence as the smoking gun).
- **Baselines:** vanilla PPR, HippoRAG node specificity (the one to beat), symmetric normalization, post-hoc degree rescaling, vector-only, BM25.
- **Ablations:** β sweep; raw degree vs. entropy-weighted; each layer alone vs. composed; robustness across extractor models (degree pathology varies by extractor).
- **Guarded failure mode:** over-punishing legitimate hubs harms multi-hop through popular entities; report per-question-type breakdowns.

Success criterion: improved multi-hop retrieval accuracy with no regression on single-hop, plus demonstrated insufficiency of seed-side correction alone.

3.4 PoC Plan and Effort

Step	Content	Effort
0	Sanity check: post-hoc rescale + entropy weighting on prebuilt HippoRAG graphs (no extraction cost)	days
1	Layer 1 kernel swap in HippoRAG 2 + diagnostics	~2 weeks
2	Layer 2 entropy statistics + integration	~1 week
3	Layer 3 facet splitting in Graphiti ingestion	~3–4 weeks
4	Full evaluation + ablations	~2–3 weeks

Main external cost: LLM API budget for extraction in Layer 3 experiments; read-path experiments reuse prebuilt graphs.

4 Future Work Packages Built on the PoC

The PoC’s two artifacts, the **leakiness metric** and the **facet abstraction**, are the keystone for the rest of the program:

- **WP2, Learned and dynamic kernels:** replace the hand-designed damping with per-node damping learned from QA supervision; condition hub transparency on the query (the “meeting” hub is legitimate for meeting queries). Add temporal composition with bi-temporal validity windows.
- **WP3, Facet lifecycle and structural maintenance:** merge/split/re-cluster facets as the graph evolves; centrality-guided consolidation and eviction; cascade re-validation of contradicted facts scoped to facets (connects to the temporal pathology group; STALE benchmark as evaluation).
- **WP4, Facets as governance and concurrency units:** facet-level access control per agent (governed shared memory), and facet-level sharding to eliminate write hotspots in multi-agent fleets, providing a substrate for the first consistency model for shared agent memory.
- **WP5, Extraction–retrieval co-design:** extraction prompting/training that avoids creating supernodes, using the leak metric as the optimization signal.
- **WP6, Engine-native operator:** push the damped kernel into a graph engine as a native query operator (natural fit for sparse-matrix engines such as FalkorDB/GraphBLAS, where the kernel is one modified matrix operation), with incrementally maintained leakiness statistics.

5 Expected Contributions

1. Characterization of structural pathology in LLM-constructed knowledge graphs (motivation, small part).
2. An entropy-weighted, hub-transparent retrieval kernel with demonstrated multi-hop gains (algorithmic contribution).
3. Write-time facet splitting as an integrity-enforcement mechanism in a production-grade agent-memory engine (systems contribution).
4. A constraint-based framing (“integrity constraints for probabilistic writers”) that organizes the wider pathology space into a coherent research program.

Abstract

AI agents increasingly store long-term memory in knowledge graphs written not by human curators but by LLMs, which systematically violate the structural invariants graph databases were built to assume. This project develops integrity mechanisms for machine-written graphs, beginning with the most acute retrieval failure: supernodes, generic hub nodes that soak up query-independent PageRank mass and manufacture false multi-hop connections. We propose an entropy-weighted, hub-transparent retrieval kernel and a write-time facet-splitting mechanism, integrated into state-of-the-art agent-memory systems and evaluated on standard multi-hop QA and agent-memory benchmarks. The resulting metric and facet abstractions form the foundation of a broader research program on self-maintaining knowledge graphs for agentic systems.